Практическая работа № 1

Разработка перечня артефактов и протоколов проекта

Дисциплина: Проектирование ПО
Автор: Студент группы [Номер]
Дата: Октябрь 2026 г.

1. ЦЕЛЬ РАБОТЫ И ОПИСАНИЕ АНАЛИЗИРУЕМОГО ПРОЕКТА

Цель: Освоить порядок определения, классификации и формализации артефактов и протоколов взаимодействия в программном проекте, научиться обосновывать их необходимость и формировать регламент управленческой и технической документации.

Паспорт анализируемого проекта: Корпоративная SaaS CRM-система «LeadFlow»

Цель проекта: Разработка мультиарендной (Multi-tenant) CRM-системы для автоматизации отдела продаж, управления воронкой сделок, сквозной аналитики и интеграции с IP-телефонией и мессенджерами.

Целевая аудитория: Малый и средний бизнес (отделы продаж, B2B-менеджеры, руководители отделов продаж, маркетологи).

Ключевые функции (минимум 5):

- 1. Управление лидами и сделкой (Канбан-доска, автоматическое распределение задач).
- 2. Интеграция с Telegram API, WhatsApp Business API и IP-телефонией (SIP) для ведения коммуникации из карточки клиента.
- 3. Конструктор аналитических отчётов (конверсия воронки, LTV, ROMI, выработка менеджеров).
- 4. Модуль автоматического выставления счетов и интеграции с 1С/клиент-банком.
- 5. Система ролевого доступа (RBAC) и аудит действий пользователей (Audit Log).

Участники проекта: Заказчик (Product Owner), Проектный менеджер (PM), Системный аналитик, Архитектор (Lead Dev), Frontend/Backend разработчики, QA-инженеры, UI/UX-дизайнер, DevOps-инженер.

2. ПЕРЕЧЕНЬ АРТЕФАКТОВ ПРОЕКТА (ПРОЕКТИРОВАНИЕ, РАЗРАБОТКА, ЭКСПЛУАТАЦИЯ)

№	Артефакт	Тип	Назначение	Кто создаёт	Кто использует	Этап ЖЦ	Формат
1	Концепция проекта (Project Charter)	Документ	Фиксация целей, бюджета, границ (Scope) и ключевых вех проекта.	PM, Заказчик	Вся команда, Стейкхолдеры	Инициация	PDF / Confluence
2	Спецификация требований (SRS)	Документ	Формализация функциональных и нефункциональных требований.	Системный аналитик	Разработчики, QA, Заказчик	Анализ требований	Confluence
3	Интерактивный UI/UX макет	Дизайн	Прототипирование пользовательских интерфейсов и CJM.	UI/UX Дизайнер	Frontend-разработчики, QA, PO	Проектирование	Figma
4	Архитектурное решение (SAD)	Схема / Doc	Описание C4-модели, компонентов, схемы БД и интеграций.	Solution Architect	Backend-разработчики, DevOps	Проектирование	Draw.io / Markdown
5	Исходный код приложения	Код	Реализация бизнес-логики CRM и пользовательского интерфейса.	Разработчики	Разработчики, DevOps, SonarQube	Разработка	Git / GitHub
6	OpenAPI / Swagger спецификация	Спецификация	Описание REST API контрактов между Frontend и Backend.	Backend разработчик	Frontend-разработчики, QA	Разработка	YAML / Swagger Hub
7	Тест-план и Тест-кейсы	Тест-документ	Набор проверок и сценариев для верификации функций CRM.	QA Engineer	QA, Разработчики	Тестирование	TestRail / Allure

3. ПЕРЕЧЕНЬ ПРОТОКОЛОВ ВЗАИМОДЕЙСТВИЯ В ПРОЕКТЕ

№	Протокол (Регламент)	Назначение и регулируемые вопросы	Участники	Периодичность	Форма фиксации
1	Протокол согласования требований	Утверждение изменений в SRS, управление Scope Creep, фиксация доп. бюджетов.	PM, Аналитик, Product Owner (Заказчик)	По событию (при изменении ТЗ)	ЭЦП / Лист согласования в Confluence
2	Протокол Code Review (Ревью кода)	Проверка соответствия кода Code Style, отсутствие уязвимостей, утверждение Pull Request.	TechLead, Разработчики	Постоянно (на каждый PR)	Запись и комментарии в GitHub PR
3	Протокол Daily Standup & Sprint Planning	Синхронизация команды, определение задач на спринт, выявление блокеров.	Вся Scrum-команда	Ежедневно / 1 раз в 2 недели	Доска Jira / Запись встречи
4	Протокол управления инцидентами / багами	Классификация критичности дефектов (Blocker, Critical), приоритезация починки.	QA, Backend/Frontend Devs, PM	По событию (обнаружение бага)	Тикет в Jira Bug Tracking
5	Протокол приёмо-сдаточных испытаний (ПСИ)	Фиксация успешного прохождения сценариев приёмки релизной версии CRM Заказчиком.	PM, QA Lead, Заказчик (PO)	В конце каждого релиза	Акт ПСИ (PDF с подписями)
6	Протокол Post-Mortem (Анализ инцидентов)	Разбор причин падения Production-серверов, выработка мер по предотвращению повторения.	DevOps, TechLead, PM, SRE	По событию (P1/P0 инцидент)	Отчёт в Confluence Post-Mortem

4. ВЗАИМОСВЯЗЬ АРТЕФАКТОВ И ПРОТОКОЛОВ (ПРИМЕРЫ)

Артефакты (результаты) и протоколы (правила) находятся в непрерывной смысловой связке. Протоколы определяют жизненный цикл артефактов:

- **Связь 1 (Требования):** Артефакт *«Спецификация требований (SRS)»* управляется *«Протоколом согласования требований»*. Без этого протокола аналитик или заказчик могут менять постановку задач «на лету», что приведет к бесконечному изменению границ проекта (Scope Creep) и срыву сроков.
- **Связь 2 (Качество кода):** Артефакт *«Исходный код»* проходит через *«Протокол Code Review»*. Изменения не попадают в ветку `main` без одобрения TechLead. Это гарантирует сохранение архитектуры (SAD) и предотвращает попадание неоптимального кода в продакшн.
- **Связь 3 (Сдача-приёмка):** Артефакты *«Отчёт о тестировании»* и *«Интерактивный UI/UX макет»* являются входными данными для *«Протокола приёмо-сдаточных испытаний (ПСИ)»*. Заказчик подписывает Акт ПСИ только на основании успешного выполнения тест-кейсов.
- **Связь 4 (Дефекты):** При обнаружении отклонений от *«OpenAPI спецификации»* QA-инженер инициирует *«Протокол управления инцидентами»*, создавая баг-трекинг тикет, который блокирует релиз до устранения.

5. ОБОСНОВАНИЕ ВЫБОРА И АНАЛИЗ РИСКОВ

Обязательные vs Специфичные артефакты: Обязательными для любого IT-проекта являются *SRS, Исходный код, Тест-кейсы*. Специфичными для нашей SaaS CRM-системы являются *OpenAPI спецификация* (из-за разделения Frontend/Backend), *Скрипты миграции БД* (из-за хранящихся данных множества клиентов) и *CI/CD конфигурации Helm/Docker* (для облачного развёртывания).

Критичность протоколов: Наиболее критичным является *Протокол согласования требований*. В SaaS-продуктах отсутствие чётко зафиксированных границ приводит к финансовым убыткам. Не менее важен *Протокол Code Review*, предотвращающий утечку данных клиентов между тенантами (Multi-tenancy security).

Анализ рисков при отсутствии ключевых артефактов и протоколов:

- **Отсутствие OpenAPI / SAD:** Backend и Frontend разработчики будут создавать интерфейсы несогласованно, что приведёт к полному переписыванию кода на этапе интеграции.
- **Отсутствие Протокола Code Review:** В релиз попадут критические уязвимости (SQL-инъекции, Cross-Tenant Data Leakage), что разрушит репутацию SaaS-сервиса.
- **Отсутствие Протокола ПСИ:** Заказчик сможет отказаться от оплаты, утверждая, что система работает не так, как он предполагал.